

Forgetting Without Restarting: Execution-State Unlearning for Stateful LLM Agents

Chao Yao¹, Yangbo Wei², Zhen Huang², Junhong Qian²,
Chenle Chen², Shaoqiang Lu², Chen Wu², Lei He^{2*}

¹Arizona State University, USA

²Eastern Institute of Technology, Ningbo, China
cyao22@asu.edu, yangforever@sjtu.edu.cn

Abstract

Long-running LLM agents are stateful: beyond the transcript they accrete compressed summaries, plaintext memory, pending tool plans, and—under every serving API—a KV cache. Yet today’s “forget” operations delete a plaintext memory record and stop, leaving every artifact derived from the revoked information intact. We formalize *execution-state unlearning*: after a forget request, the agent must behave as if it had never observed the target. Modeling the runtime as a deterministic transition system, we prove that the pre-target trajectory prefix is shared with this counterfactual world for free, that the post-target suffix is irreducibly tainted without token-level attribution, and that exact unlearning requires at least $T - \tau + 1$ recomputed transitions, where τ is the target’s injection step. *Provenance-Guided Selective Replay* attains this bound as a cross-layer contract spanning prompt, compressed memory, and cache: a provenance graph locates the injection point, checkpoint restoration reduces to *cropping* the KV cache, and sanitized replay regenerates the counterfactual suffix. Audited with elicitation, stochastic, and string-free behavioral tests across three agent suites, nine baselines, and three model families, memory deletion leaves leakage unchanged, instruction-based forgetting collapses under elicitation (Leak@probes = 1.00), and source redaction still *acts* on a revoked preference in 80% of episodes—while selective replay is indistinguishable from a full reset at up to $9\times$ fewer recomputed tokens.

Introduction

Within months of release, agent frameworks such as OpenClaw (Steinberger 2026) and Hermes Agent (Nous Research 2026) accumulated hundreds of thousands of deployments as *always-on personal agents* that run for weeks, operate tools, and remember their users. What makes them useful is precisely that they are *stateful*: a modern runtime layers the transcript; *context compaction* that rewrites older turns into model-authored summaries; plaintext long-term memory re-injected at session start (Packer et al. 2023; Chhikara et al. 2025); tool traces and pending plans; and, beneath all of these, the KV cache—universal serving infrastructure reused across requests by every engine and commercial API (Kwon et al. 2023; Zheng et al. 2024; Gim et al. 2024). Whatever enters an agent’s context is compressed into summaries, distilled into plans, persisted into memory, and materialized as cached tensors (Figure 1). This collides with an equally basic

*Corresponding author.

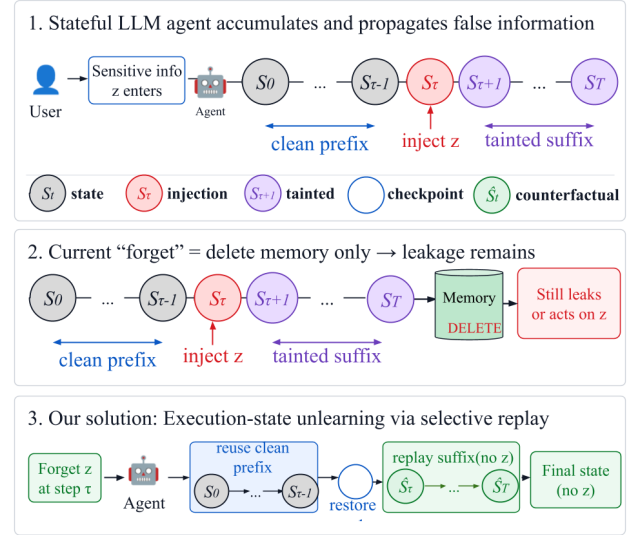


Figure 1: Execution-state unlearning at a glance. A target z entering at step τ splits the trajectory into a clean prefix and a suffix whose summaries, plans, and cache are all tainted (1). Deleting the memory record leaves that suffix intact—the agent still leaks or acts on z (2). Selective replay restores $\hat{S}_{\tau-1}$ by *cropping* the cache, then replays the sanitized suffix to $\hat{S}_T$ (3).

requirement: sometimes the agent must *un-see* something—a user revokes consent for a home address mid-session (European Parliament and Council of the European Union 2016), a secret is pasted by accident, or an indirect injection plants content inside a fetched page or tool result (Greshake et al. 2023). The last case is the common one and the user never witnesses it: the agent silently reads the injected content, folds it into its state, and moves on; the forget trigger, when it comes, comes from a detector or operator *after the fact*. Yet deployed stacks offer only a forgetting affordance that operates on *plaintext at a single layer*: delete the memory record, edit the Markdown file, drop the message from retrieval. The industry has equated forgetting with un-indexing.

We show systematically that this equation fails, and that the failure is invisible to the string-matching evaluations used to certify it. On three agent suites instrumented with mem-

ory injection, compaction, and tool use (Wu et al. 2025; Lu et al. 2024; Debenedetti et al. 2024), deleting the persistent memory record leaves leakage *exactly* unchanged from doing nothing (0.86–1.00 any-leak): the target survives in the session’s derived state. An instruction to forget looks far better on a single task-shaped probe, but under a six-probe elicitation audit the same state yields the target with probability 1.00—merely suppressed. Source redaction fails more subtly: the model’s own summary re-encodes the target in paraphrase, beyond any forbidden-string list, and in a behavioral test the redacted agent still *acts* on a revoked preference in 80% of episodes while emitting the string zero times. String metrics certify precisely the methods that fail. Even information-flow control (Costa et al. 2025), which blocks tainted *future* flows, cannot clean state already contaminated: an IFC-only baseline leaks at the no-forget rate.

What should “forget” mean for a running agent? We argue for a counterfactual criterion: future behavior must be indistinguishable from a twin agent that *never observed* the target. Modeling the runtime as a deterministic transition system, we define the counterfactual trajectory induced by deleting the target z from the observation stream at its injection step τ , and call an operator an exact *execution-state unlearner* if it maps the real final state to the counterfactual one; unlike parameter unlearning (Cao and Yang 2015; Bourtole et al. 2021; Maini et al. 2024), the edited object is non-parametric runtime state, where exactness is attainable. The formalism yields sharp structure: a prefix-sharing lemma makes the first $\tau - 1$ counterfactual steps *free* (the clean prefix is literally a prefix of the contaminated cache, so restoring it is a crop); a taint lemma shows that without token-level attribution every artifact at or after τ is unsalvageable—computation cannot be edited, only replayed. A splicing theorem then proves checkpoint-and-replay reconstructs the counterfactual state exactly, with a matching $T - \tau + 1$ lower bound: forgetting cost is governed by the counterfactual divergence, not the session length.

We realize this as *Provenance-Guided Selective Replay*, an auditable cross-layer contract from prompt to compressed memory to KV cache: an artifact-level provenance graph recorded during execution, sparse metadata-only checkpoints whose restoration is a cache crop, and sanitized replay with shadow-executed, deduplicated consequential tools. Because string matching cannot certify forgetting, we audit with *Leak@probes* (six elicitation probes), *Leak@5* (stochastic samples), a string-free *behavioral-extraction* suite, and counterfactual-action divergence, each anchored to a measured false-positive floor. Selective replay sits at the floor on every axis while recomputing up to $9\times$ fewer tokens than a full reset, with cost tracking the proven $T - \tau + 1$ line ($R^2 \approx 1$); results reproduce across Llama-3.1-8B, Qwen2.5-7B, and Mistral-7B.

Our contributions: **(1) Problem:** execution-state unlearning for stateful LLM agents, formalized via counterfactual equivalence over reconstructible runtime state—the layer today’s forget operations silently skip. **(2) Theory:** the prefix-sharing and certifiable-taint-boundary lemmas, an exact splicing theorem, and $T - \tau + 1$ optimality. **(3) System:** Provenance-Guided Selective Replay, composing

provenance, crop-as-restore checkpoints, and side-effect-safe replay into one forgetting contract. **(4) Audits and evidence:** elicitation, stochastic, and string-free behavioral audits with measured floors; nine baselines on three suites and three model families—every deployed-style forget fails at least one audit; selective replay matches a full reset at a fraction of its cost.

Related Work

Machine unlearning. Unlearning classically removes training data’s influence from *model parameters*, exactly by retraining from sharded checkpoints (Cao and Yang 2015; Bourtole et al. 2021) or approximately by fine-tuning (Eldan and Russinovich 2023), with benchmarks surveyed for LLMs (Maini et al. 2024; Liu et al. 2025); approximate unlearning is notoriously hard to verify. Our setting inverts this: the object is the agent’s *non-parametric execution state*, whose transition function is replayable, so *exact* unlearning is attainable and certifiable. Conceptually, checkpoint-and-replay is the runtime analogue of SISA’s shard-and-retrain (Bourtole et al. 2021), except that causality gives the shard boundary (τ) for free.

Agent memory systems. Long-horizon agents externalize state into managed memory: paged context (Packer et al. 2023), extracted fact stores (Chhikara et al. 2025), and, in deployed frameworks, plaintext Markdown or SQLite memories with compaction (Steinberger 2026; Nous Research 2026). All expose deletion of a stored record; none propagate it into the live session’s derived artifacts or cache, and memory benchmarks (Wu et al. 2025) evaluate recall, not revocation. Our episodes exercise exactly these abstraction layers, and their delete operation is baseline B1—behaviorally a no-op.

KV-cache reuse and serving. Prefix caching is universal serving infrastructure: paged attention (Kwon et al. 2023), radix-tree prefix sharing (Zheng et al. 2024), and modular attention reuse (Gim et al. 2024) all reuse attention states keyed on byte-identical prefixes. This machinery is built for *reuse*, not *revocation*: it provides no statement about what a cached suffix still encodes. We run the same mechanism in reverse—Lemma 1 makes *cropping* a cache a certified restoration operator—and add what caching cannot: a provenance-backed guarantee of what was regenerated and why.

Agent security: prevention, detection—and no remediation. Indirect prompt injection (Greshake et al. 2023; Debenedetti et al. 2024) has produced two defense families. *Prevention by design* constrains what untrusted content can do before it does it: instruction-hierarchy training (Wallace et al. 2024), quarantine and plan-then-execute patterns (Beurer-Kellner et al. 2025), capability policies over extracted data flows (CaMeL; Debenedetti et al. 2025), and information-flow labels with deterministic sink gating (FIDES; Costa et al. 2025). *Detection* flags injected content via classifiers, model-internal features, or localization of the injected span (Jia et al. 2026). Both leave the same

gap: prevention is imperfect, and detection is routinely *asynchronous*—by the time a flag fires, the agent has already read the content, folded it into summaries, plans, and cache, and moved on. What happens to that session is unaddressed: IFC constrains future flows but cannot clean resident state (our B8 leaks at the no-forget rate), and no defense above offers a remediation primitive. We supply the recovery half: a detector’s verdict is exactly the (z, s) input Algorithm 1 consumes, and composing IFC with splicing (B9) enforces sink policies over a runtime that no longer contains the target.

Method: Forgetting as Counterfactual Trajectory Splicing

Formalizing the runtime as a deterministic transition system makes the “twin agent that never saw the target” a well-defined object—the counterfactual trajectory. Causality guarantees the two trajectories coincide pointwise before the target enters, so forgetting reduces to a *splicing* problem: reuse the common prefix already computed, and re-enact only the post-divergence suffix in the world without the target. The minimal cost of forgetting is thus governed by the *counterfactual divergence* $T - \tau$, not the session length T ; our method realizes this bound as an executable system.

The Runtime as a Deterministic Transition System

Let the runtime state be $R_t \in \mathcal{R}$ (comprising the KV cache, working memory, uncommitted tool plans—all reconstructible components), and let $o_1, \dots, o_T \in \mathcal{O}$ be the external observation stream (user inputs, memory injections, tool returns). A session is the trajectory

$$R_t = F(R_{t-1}, o_t; \theta), \quad t = 1, \dots, T, \quad R_0 = R_{\text{init}}, \quad (1)$$

where $F : \mathcal{R} \times \mathcal{O} \rightarrow \mathcal{R}$ is determined jointly by the model’s forward computation and the agent scaffold. so transition t consumes o_t and produces R_t , and R_T is the final state. The forget target z first enters through an observation at step $\tau \geq 1$: $z \in o_\tau$ and $z \notin o_t$ for all $t < \tau$.

Define the *counterfactual observation stream* $o^{-z} = (o_1^{-z}, \dots, o_T^{-z})$ with $o_t^{-z} = o_t$ for $t \neq \tau$ and $o_\tau^{-z} = o_\tau \setminus \{z\}$. With $R_0^{-z} = R_{\text{init}}$ and $R_t^{-z} = F(R_{t-1}^{-z}, o_t^{-z}; \theta)$, this induces the *counterfactual trajectory* $\{R_t^{-z}\}$ —the parallel world in which the agent never saw z .

Definition 1 (Counterfactual-equivalent forgetting). *An unlearning operator $U : \mathcal{R} \times \mathcal{Z} \rightarrow \mathcal{R}$ is exact iff $U(R_T, z) = R_T^{-z}$ (deterministic decoding); under stochastic decoding this relaxes to ε -consistency of future behavior distributions, $D(P_A(\cdot | U(R_T, z)), P_A(\cdot | R_T^{-z})) \leq \varepsilon$.*

The definition packages three intuitive requirements at once: the target is no longer accessible (the counterfactual never contained z); derived influence is removed (the counterfactual summary/plan never depended on z); and non-target utility is preserved (all other observations are kept verbatim). Note also what it does *not* assume: who asks. The operator consumes only a revocation event (z, s) naming the target and its source artifact—raised by the user, the platform, or an injection detector flagging a tool observation *after* the agent has processed it; in that last, common case the

user never saw z , and provenance, not human recollection, locates τ . The operator realizing U works over the recorded runtime—observation log, provenance graph, checkpoints, environment snapshots—the computational model made explicit in Corollary 1.

Two Lemmas: a Free Prefix and a Stubborn Suffix

Lemma 1 (Prefix sharing). *For all $t \leq \tau - 1$, $R_t = R_t^{-z}$.*

Proof. By induction on t . *Base:* $R_0 = R_{\text{init}} = R_0^{-z}$. *Step:* if $R_{t-1} = R_{t-1}^{-z}$ for some $t \leq \tau - 1$, then $R_t = F(R_{t-1}, o_t; \theta) = F(R_{t-1}^{-z}, o_t; \theta) = F(R_{t-1}^{-z}, o_t^{-z}; \theta) = R_t^{-z}$, using the induction hypothesis and $o_t = o_t^{-z}$ for $t < \tau$. $\square$

The proof is trivial; the corollary is not: *the first $\tau - 1$ counterfactual steps have already been computed, for free, by the real execution.* The clean prefix is not a cache-optimization trick—it is mathematically the shared part of the two worlds, and any scheme that discards it (e.g., a full reset) recomputes history on which the trajectories are identical.

Lemma 2 (Taint monotonicity and the certifiable boundary). *Record runtime dependencies as a directed graph $G_t = (A_t, E_t)$, with A_t the artifacts produced up to t and E_t the recorded data-flow edges; define the influence set as the reachability closure $I_t(z) = \{a \in A_t : z \rightsquigarrow_{G_t} a\}$. Then: (i) monotonicity: $I_t(z) \subseteq I_{t+1}(z)$ for all t ; (ii) certifiable boundary: absent per-token influence attribution (i.e., without decomposing F into selective reads of state components), the maximal artifact set certifiably independent of z is exactly the prefix output $\{a : \text{turn}(a) < \tau\}$.*

Proof. (i) Execution only appends nodes and edges: $A_t \subseteq A_{t+1}$, $E_t \subseteq E_{t+1}$, and reachability is monotone in the edge set. (ii) Artifacts with $\text{turn}(a) < \tau$ are generated by the shared prefix of Lemma 1, so independence from z is directly certifiable. Conversely, any artifact with $\text{turn}(a) = t \geq \tau$ is generated by a transition that reads the full state R_t , and $z \rightsquigarrow R_\tau \rightsquigarrow \dots \rightsquigarrow R_t$, so the conservative graph contains a path $z \rightsquigarrow a$, i.e., $a \in I_t(z)$; excluding that path would require proving the invocation of F did not use the z -dependent components of R_t —exactly the per-token attribution capability we excluded. Hence the certified-clean set is $A_t \setminus I_t(z) = \{a : \text{turn}(a) < \tau\}$. $\square$

Lemma 2 is the theoretical root of “deletion $\neq$ forgetting”: once read, z ’s influence propagates along the reachability closure into summaries, plans, and pending tool calls; local edits can remove nodes of $I(z)$ but cannot reverse computation that has already happened—*computation cannot be edited, only replayed*. Here, non-reusability refers to the *original* post-target KV and model-derived runtime states: no state at or after τ may be carried over. It does not mean that every post-target token must be re-decoded. Content whose value is fixed in the counterfactual world—recorded observations under Assumption (A2), and, when an attribution oracle stronger than the one Lemma 2(ii) assumes away certifies it, model turns independent of z —may be re-materialized by

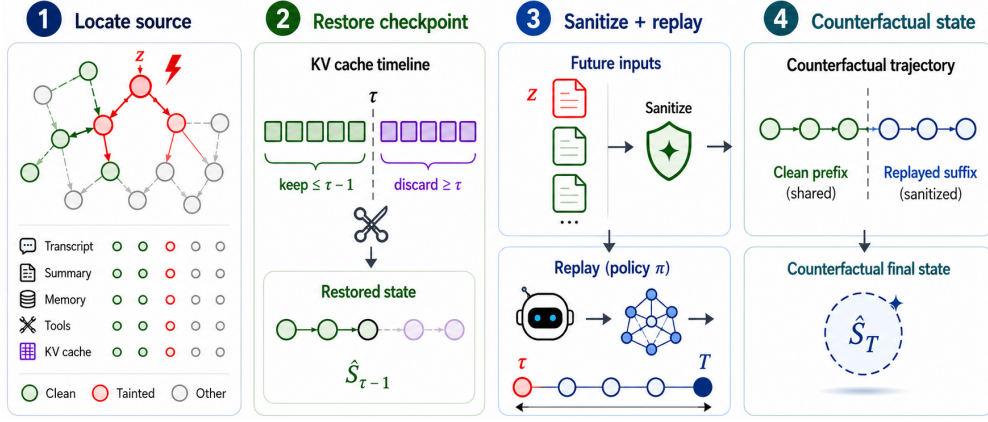


Figure 2: Provenance-Guided Selective Replay (Alg. 1). A reachability query returns τ and the taint closure (I); the KV timeline is cropped at $\tau-1$, the free prefix of Lemma 1 (2); the suffix is reconstructed with z dropped and all else verbatim (3); the splice ends at $\hat{S}_T = R_T^{-z}$ (4), Theorem 1.

prefill on the freshly reconstructed cache; this is still reconstruction, since the original post-target KV representation is never reused.

Lemma 2 also separates two kinds of selectivity. Direct state *reuse* is confined to the time dimension: only the clean prefix survives, and there is no per-item triage of the suffix’s KV. *How* each reconstructed transition is recomputed is a separate question: counterfactually fixed content can be replayed by prefill, while genuinely model-derived content must be decoded again.

The Splicing Theorem and Optimality

Lemma 1 says the clean prefix can be reused directly; Lemma 2 says the original suffix state cannot be retained and must be reconstructed in order. This state-level reconstruction does not require autoregressively regenerating every recorded token: content fixed under Assumption (A2) may be replayed through prefill. Their combination is the method:

Theorem 1 (Splicing equivalence). *Let a checkpoint exist at $\tau - 1$ (or any earlier clean boundary). Define the replayed trajectory $\tilde{R}_{\tau-1} = R_{\tau-1}$, $\tilde{R}_t = F(\tilde{R}_{t-1}, \tilde{o}_t; \theta)$ for $t \geq \tau$, with sanitized observations $\tilde{o}_t$. If (A1) decoding is deterministic (or the randomness source is fixed); (A2) sanitized observations agree with the counterfactual ones, $\tilde{o}_t = o_t^{-z}$ for all $t \geq \tau$; and (A3) no committed external side effects exist after τ (the environment can be restored from a snapshot so the tool observations in (A2) are reproducible); then $\tilde{R}_t = R_t^{-z}$ for all $t \geq \tau - 1$; in particular $\tilde{R}_T = R_T^{-z}$, i.e., the splicing operator is an exact unlearner in the sense of Definition 1.*

Proof. Induction on t . *Base* ($t = \tau - 1$): $\tilde{R}_{\tau-1} = R_{\tau-1} = R_{\tau-1}^{-z}$ by the restore operation and Lemma 1. *Step:* if $\tilde{R}_{t-1} = R_{t-1}^{-z}$ for some $t \geq \tau$, then $\tilde{R}_t = F(\tilde{R}_{t-1}, \tilde{o}_t; \theta) = F(R_{t-1}^{-z}, \tilde{o}_t; \theta) = F(R_{t-1}^{-z}, o_t^{-z}; \theta) = R_t^{-z}$, where the second and third equalities use, respectively, (A1) to make F single-valued (otherwise pointwise equality is not even well

posed) and (A2)/(A3) to guarantee the step- t tool observation attains o_t^{-z} during replay. $\square$

Corollary 1 (Recomputation lower bound and optimality). *In the computational model where an operator may only (a) read the stored real trajectory $\{R_t\}_{0 \leq t \leq T}$, observation stream, and derived metadata (provenance graph, checkpoints, environment snapshots), or (b) invoke F to advance a state, any exact unlearning operator must invoke F at least $T - \tau + 1$ times in the worst case. The splicing operator (Algorithm 1) invokes it exactly $T - \tau + 1$ times and is therefore optimal under the conservative taint model; the gain over a full reset (T invocations) is $T/(T - \tau + 1)$.*

Proof sketch. *Lower bound:* exactness requires outputting R_T^{-z} . When z has nonzero influence, $R_t^{-z} \neq R_t$ for all $t \geq \tau$ in the worst case, so no state on the counterfactual suffix is stored and route (a) is unavailable; each invocation of route (b) advances the counterfactual trajectory by one step, and by Lemma 1 the only stored state lying on it is at most $R_{\tau-1}$. Advancing from $R_{\tau-1}^{-z}$ to R_T^{-z} takes $T - (\tau - 1)$ invocations. *Upper bound:* Algorithm 1 replays from $\tilde{R}_{\tau-1}$ in exactly $T - \tau + 1$ steps, exact by Theorem 1. $\square$

Corollary 1 yields a testable prediction: forgetting cost equals the post-target *suffix length* $T - \tau + 1$, not the session length T —the later the target arrives, the closer forgetting is to free; the ablations below verify it. The bound counts sequential state *transitions*, not autoregressively decoded tokens: content that is fixed within a replayed transition may be re-materialized by prefill, so token-level cost can fall below the transition count without contradicting the lower bound. (Since Theorem 1 makes equality a *constructive* guarantee under deterministic decoding, the experiments emphasize efficiency and distributional consistency under stochastic decoding rather than treating agreement as a discovery.)

From Theorems to System

Each theoretical object maps to a system component (Figure 2), answering respectively *where* to splice, *what* to splice,

Algorithm 1: Provenance-Guided Selective Replay

Require: runtime R_T , target z , provenance graph G , checkpoints $\mathcal{C}$, recorded observations $\{o_t\}_{t=1}^T$
Ensure: counterfactual-equivalent runtime $R' = R_T^{-z}$
1: $s \leftarrow \text{LocateSource}(G, z)$ $\triangleright$ source artifact of z
2: $\tau \leftarrow \text{FirstEntry}(G, s)$ $\triangleright$ first entry boundary
3: $I(z) \leftarrow \text{TaintClosure}(G, s)$ $\triangleright$ invalidate closure
4: $c^* \leftarrow \arg \max\{c \in \mathcal{C} : \text{boundary}(c) < \tau\}$
5: $R \leftarrow \text{Restore}(c^*)$ $\triangleright$ crop KV to c^* ; load env snapshot
6: **for** $t = \text{boundary}(c^*) + 1 \dots T$ **do**
7: $\tilde{o}_t \leftarrow \text{Sanitize}(o_t, z)$ $\triangleright$ remove z ; keep the rest (A2)
8: $\tilde{o}_t \leftarrow \text{ReplayTools}(\tilde{o}_t)$ $\triangleright$ shadow exec + dedup (A3)
9: $R \leftarrow F(R, \tilde{o}_t; \theta)$ $\triangleright$ regenerate derived artifacts
10: **end for**
11: **return** $R' \leftarrow R$

and *how*:

(a) **Provenance graph G — causal reachability, materialized (where).** During execution we record artifact-level data flow: memory/tool field $\rightarrow$ prompt block $\rightarrow$ model turn $\rightarrow$ reply/plan $\rightarrow$ tool call $\rightarrow$ observation $\rightarrow$ summary write-back, each artifact carrying (id, type, parents, source_ids, token_span, turn, committed). On a forget request, one reachability query returns the injection point τ and taint closure $I(z)$. We deliberately do *not* attempt token-level attribution: the graph records only dependencies that actually occurred—conservative but certifiable (Lemma 2ii).

(b) **Sparse checkpoint set $\mathcal{C}$ — splice points, materialized (what).** At semantic boundaries (session start, turn boundaries, before memory injections, consequential tool calls, and compactions) we register checkpoints holding only metadata (token offset, cache handle, environment-snapshot ID, prompt manifest)—no tensor copies. By Lemma 1, *cropping is restoring*.

(c) **Sanitized replay — re-enacting the counterfactual suffix (how).** Read-only/deterministic tools are re-executed from the environment snapshot or replayed from recorded observations (realizing A2/A3); consequential tools are shadow-executed during replay with call-ID deduplication, so real side effects never fire twice.

Algorithm 1 assembles the pipeline: lines 1–3 are provenance queries (locate τ , invalidate $I(z)$); lines 4–5 are the $O(1)$ checkpoint restore (Lemma 1); lines 6–10 re-enact the counterfactual suffix (the construction of Theorem 1); total recomputation $T - \tau + 1$ attains the bound of Corollary 1.

Experiments

Implementation and Setup

Runtime harness. We implement this transition system on HuggingFace Transformers with explicit KV management. A KVEngine exposes the three primitives the theory needs: *prefill*, *generate*, and *crop*. Every block, turn, tool call, observation, and summary is a `RuntimeBlock` carrying the provenance tuple above; oracle source-ID propagation (a turn generated while the target is resident inherits its source ID) populates the `ArtifactGraph`, whose taint queries implement lines 1–3 of Algorithm 1. A

`CheckpointStore` keeps metadata-only handles at session start, turn boundaries, and before the target; `Restore` is a *crop* call, granularity ablated below.

Episodes. Each episode is a three-phase turn script—*setup* (clean prefix), *acquisition* (the target enters via memory injection or tool observation), *contamination* (≥ 1 model turns folding the target into an answer and, per suite, a *summary* and/or pending *tool plan*)—re-run with the target excluded to produce the counterfactual reference. Episodes are converted from **LongMemEval** (Wu et al. 2025) ($n=100$, memory-injected facts), **ToolSandbox** (Lu et al. 2024) ($n=100$, tool-observed identifiers), and **AgentDojo** (Debenedetti et al. 2024) ($n=80$; slack/workspace/banking/travel, 20 each); the tool-observation channel instantiates the detector-triggered case above: the target arrives inside a tool result the user never sees, and the forget request names the flagged observation. Future queries are task-shaped and solicit the target (e.g., “schedule an appointment *near my home*”).

Methods. All methods branch from the *same* contaminated base state (cache cloned), so comparisons are paired: **B0** No-Forget; **B1** Memory-Delete (remove the persistent record, session untouched—what deployed stacks do); **B2** Forget-Instruction (append “forget z ”); **B3** Source-Redaction (drop the source block, keep derived artifacts); **B4** Sanitize-no-Replay (drop source + descendants, regenerate nothing); **B5** Full-Reset (the counterfactual reference R_T^{-z}); **B5'** Full-Reset + prefix cache (control isolating how much of B7’s saving a generic cache recovers); **B6** Sanitized-Rebuild (string-redact the transcript, rebuild); **B7** **Selective-Replay** (Algorithm 1); **B8** FIDES-style IFC (Costa et al. 2025) (sink policy over identifier-type tool arguments, no state cleanup) and **B9** IFC + replay.

Models and decoding. Primary model: Llama-3.1-8B-Instruct on one RTX 4090; cross-family replication on Qwen2.5-7B-Instruct and Mistral-7B-Instruct-v0.3 (Grattafiori et al. 2024; Yang et al. 2024; Jiang et al. 2023). Main tables use temperature 0; stochastic audits use $k=5$ samples at sampling temperature 0.7 under matched or independent seeds as noted (T denotes session length throughout).

Metrics and audits. *Leakage* is scored over the final answer, tool arguments, and memory write-backs: *exact* (normalized string variants), *action* (target in a tool argument, by sink class: routing / selection / free-text), and *any*. *CAD* (counterfactual action distance) scores over-deletion: tool-choice mismatch plus argument distance vs. the B5 reference. *Utility* checks preserved non-target facts; *efficiency* reports reused vs. recomputed tokens and latency. Since single-probe string matching is a lower bound, we add three audits, each disciplined by a measured false-positive floor (every probe also runs against B5; probes with nonzero floors are dropped—this excluded an LLM judge, floor 0.34): **Leak@probes** (six probes: task, direct, think, introspect, enumerate, cued-completion), **Leak@5** (5 stochastic samples), and a **behavioral-extraction** suite (below). Significance is paired throughout: exact McNemar (binary), Wilcoxon signed-rank (continuous).

Method	Any-leak ↓			CAD ↓	Recomp ↓
	LME	TS	AD	(avg)	(avg tok)
B0 No-Forget	0.86	1.00	0.97	0.37	36
B1 Memory-Delete	0.86	1.00	0.97	0.37	36
B2 Forget-Instruction	0.38	0.72	0.00 [†]	0.39	55
B3 Source-Redaction	0.84	0.94	0.39	0.33	200
B4 Sanitize-no-Replay	0.00	0.00	0.00	0.22	36
B6 Sanitized-Rebuild	0.00	0.00	0.00	0.39	220
B5 Full-Reset (ref)	0.00	0.00	0.00	0.00	1235
B5' +PrefixCache	0.00	0.00	0.00	0.00	202
B7 Selective-Replay	0.00	0.00	0.00	0.00	133

Table 1: Headline results (temperature 0; LME $n=100$, TS $n=100$, AD $n=80$; CAD/Recomp averaged over suites). Sessions run 12 turns past the target, half of them independent of it ($indep = 0.5$); Recomp counts tokens a method must recompute. [†]AD’s task query never solicits the target; cf. B2 = 1.00 in Table 2.

Method	Leak@probes ↓			Leak@5 ↓
	AD	LME	TS	TS
B0 No-Forget	1.00	1.00	1.00	1.00
B2 Forget-Instruction	1.00	1.00	1.00	0.70
B3 Source-Redaction	0.73	0.80	0.97	1.00
B6 Sanitized-Rebuild	—	—	—	0.00
B8 FIDES-style IFC	—	—	—	1.00
B5 Full-Reset (floor)	0.00	0.00	0.00	0.00
B7 Selective-Replay	0.00	0.00	0.00	0.00

Table 2: Adversarial audits ($n=30$ /suite). Leak@probes: leaked under *any* of six probes against the same post-unlearning state; Leak@5: any of 5 samples at temperature 0.7 (ToolSandbox). B5 = measured false-positive floor.

Deletion is Not Forgetting

Table 1 makes the negative claim precise. **B1 equals B0 in every cell**: deleting the persistent record changes nothing—the target survives in answer, summary, plan, and cache ($p=1.0$ vs. B0). **B2** suppresses leakage but leaves the value resident (*Elicitation*, below). **B3** leaks through derived artifacts: the model’s summary and plan re-emit the target (0.84/0.94 any-leak). On ToolSandbox, 98% of B0/B1 leaks flow through *routing* arguments, steering side effects. **B4** achieves string-clean state by amputation but removes artifacts present in the counterfactual (CAD 0.22): over-deletion, not forgetting. **B8** (IFC only) blocks identifier sinks yet leaks at the B0 rate through free text (0.92–1.00); adding replay (**B9**) drops it to zero ($p<0.001$): state cleanup and flow control are orthogonal. **B7** matches B5 exactly (any-leak 0, CAD 0, agreement 1.0) while recomputing $9.3\times$ fewer tokens than a full reset (133 vs. 1235; $p<10^{-3}$) and $1.5\times$ fewer than B5’, the prefix-cached control that recovers the same prefill saving but re-decodes the whole suffix—that residual gap is what provenance buys, and it scales with *indep*.

Elicitation and Stochastic Audits

Table 2 shows why single probes mislead. **B2’s Leak@probes is 1.00 on all three suites**: an introspection probe (“what were you told to forget?”) alone re-licenses the

Method	avoid	residue	p	says it
B0 No-Forget	1.00	+0.68	< .001	0.00
B1 Memory-Delete	1.00	+0.68	< .001	0.00
B2 Forget-Instruction	1.00	+0.68	< .001	0.00
B3 Source-Redaction	0.80	+0.48	< .001	0.00
B6 Sanitized-Rebuild	1.00	+0.68	< .001	0.00
B5 Full-Reset (ref)	0.32	—	—	0.00
B7 Selective-Replay	0.32	+0.00	1.0	0.00

Table 3: Behavioral extraction (30 preference episodes $\times$ 2 counterbalanced orders). The agent picks between two near-equivalent providers, one excluded by a *revoked* preference; “avoid” = rate of picking the other, “says it” = string leakage of the revoked reason. The zero point is *measured* (B5 = 0.32), not assumed.

value at 0.87–1.00, and enumeration and chain-of-thought probes surface it where a direct question does not: the instruction leaves the value in state and commands silence. B3’s paraphrased derivations yield 0.73–0.97; B7 sits at the B5 floor: zero under every probe, including *cued* completion, and zero on Leak@5. The audit also resolves Table 1’s AgentDojo anomaly: B2’s task-probe 0.00 reflects a query that never solicits the target, not the method; safety claims must rest on Leak@probes.

Behavioral Extraction: Influence Without Strings

Every other leakage number here is a string match, so a method that stops *saying* the target scores 0.00 whether or not it still shapes what the agent *does*; Table 3 separates the two via a *revoked preference* (an exclusion) the agent need never state to act on. **B0, B1, B2 and B6 all score 0.00 on string leakage yet act on the revoked preference in 100% of episodes** (residue +0.68, $p<0.001$); B7 and B5’ match the reference (+0.00). A variant handling the redaction base-lines an oracle (the excluded brand added to the forbidden list) is instructive: B6, which redacts *every* artifact including the model-authored summary, drops to the floor (0.35, n.s.); B3, which keeps derived artifacts, stays at 0.80. The honest reading: string redaction works *only if* it reaches every derived artifact *and* the exact string is known in advance; a preference has neither property—the model paraphrases it into its own notes. Replay needs no such assumption.

Exactness, Efficiency, and Generality

Exactness (Definition 1). Under matched seeds at temperature 0.7, B7 reproduces the B5 reference *token-for-token* in 450/450 draws ($\epsilon=0$); B6 manages 1%. Under independent seeds—the meaningful distributional test—no divergence from the B5 self-sampling floor is detectable for B7 on any of the four distances (tool consistency, tool TV, n -gram, embedding; Wilcoxon $p > 0.5$ throughout), whereas the same test flags B6 (tool TV $p<10^{-3}$; Figure 3). Non-significance is not proof of equivalence; B6 is the positive control showing the test has power here, and formal equivalence testing (TOST against a pre-registered margin) is future work.

The $T - \tau + 1$ law (Corollary 1). Sweeping the injection step τ from the first turn to the last, B5’s recomputation

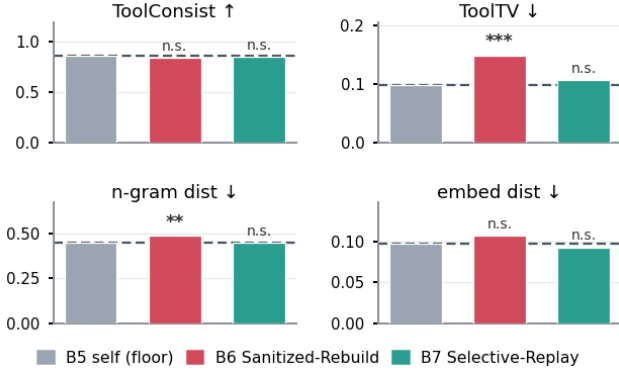


Figure 3: Behavioral distribution under stochastic decoding (pooled $n \approx 90$; $k=5$ samples at temperature 0.7, independent seeds; dashed = B5 self-sampling floor). Stars: Wilcoxon vs. floor (** $p < .01$, *** $p < .001$). No B7 panel diverges detectably; B6’s stars show the test has power.

stays flat while B7’s descends linearly in τ ($R^2=1.000$ on all three suites; LongMemEval: 1218→162 tokens vs. B5’s constant 1330)—tracking the post-target suffix length, not the session length. The second axis is *indep*, the fraction of post-target work causally independent of z —what moves B7 and B5’ apart. Real sessions carry such work in bulk: a skill card or document loaded and never used, a routine tool poll, an unrelated sub-task. Those turns are byte-identical counterfactually, so B7 *re-prefills* them (parallel) and *re-decodes* only the target-dependent remainder, while B5’ recovers the same prefill saving but re-decodes the whole suffix. B7’s decode cost therefore falls linearly with *indep* ($R^2=1.00$) while B5’ stays flat at 545–610 tokens; at full independence B7 decodes 114–147 tokens, 2.9–3.7 vs. 11.5–12.7 s p50 (prefix caching alone: 1.01 $\times$). At *indep* = 0 they coincide exactly—the honest degenerate case—with any-leak and CAD at the B5 floor throughout. This saving relies on an attribution oracle stronger than Lemma 2(ii)’s conservative model; it reduces token cost *within* transitions, not their number, leaving Corollary 1 intact. B7 further provides cache-residency independence, snapshot restoration, and a certifiable audit trail.

Cross-model. The two load-bearing findings—B3 still leaks, B7 reaches the clean reference cheaply—reproduce on both other families (280 paired episodes each; B3 any-leak 0.84–1.00, B7 all-zeros, 2.9–4.3 $\times$ savings). The one anomalous cell in the matrix (Llama’s B3 = 0.39 on Agent-Dojo) is model-specific ($B3 \geq 0.92$ elsewhere)—a reason single-model unlearning evaluations mislead.

Ablations

Invalidation boundary (Lemma 2 is not pessimism). Could one keep the suffix KV and excise just the target’s span? Table 4 says no: span-only excision still leaks in up to 93% of episodes—the surrounding KV was *computed while attending to the target*—and its CAD (0.47) is *worse than no forgetting at all* (0.38), since deleting mid-context positions corrupts state the model misreads. Each escalation fixes one

Invalidation reach	Any-leak ↓			CAD ↓
	AD	LME	TS	(avg)
None (B0)	0.97	0.83	1.00	0.38
Target span only	0.13	0.63	0.93	0.47
Span + descendants (B4)	0.00	0.00	0.00	0.23
+ replay (B7)	0.00	0.00	0.00	0.00
Full reset (B5)	0.00	0.00	0.00	0.00

Table 4: Invalidation-boundary ablation ($n=30$ /suite). Span-only excision is unsafe *and* damages utility (CAD worse than no forgetting); safety needs the descendant closure, equivalence additionally needs replay.

failure mode: the descendant closure (B4) zeroes leakage but leaves the runtime missing artifacts the counterfactual would have (CAD 0.23); only replay reaches equivalence. This is the empirical face of Lemma 2, and the depth sweep gives the matching *necessity* argument for provenance: B3 is clean at derivation depth 0 (any-leak 0.00) and degrades to 0.33–0.97 as an answer, plan, and summary stack on top; B7 holds any-leak = CAD = 0 at every depth.

Checkpoint granularity (an efficiency knob, not a correctness one). Sweeping checkpoint policies from *every_turn* to *session_start* (one checkpoint, no reusable pre-target prefix) leaves any-leak and CAD flat at zero: by Theorem 1, replay from an earlier clean boundary is equally exact, merely longer. Granularity buys only prefill reuse (~ 1000 tokens, 0.1–0.15 s) at negligible meta-data cost (≤ 4.4 KB); B7 keeps its latency advantage even at *session_start* (7.8 vs. 12.2 s), since it comes from provenance-guided *re-prefilling*, which needs the artifact graph, not the checkpoint.

Target position (leakage is position-invariant; cost is not). Moving the injection early/middle/late leaves every method’s leakage and CAD essentially unchanged—the target contaminates the session wherever it sits—while B7’s cost ratio to a full reset falls from 0.91 to 0.14: the $T/(T - \tau + 1)$ profile of Corollary 1.

Conclusion

Stateful agents broke the equation between deleting a record and forgetting it: once read, information propagates into summaries, plans, and cached tensors that forget operations never touch. Defining forgetting as counterfactual equivalence—*exactly* achievable at the runtime layer, where splicing the clean prefix to a replayed suffix costs $T - \tau + 1$ transitions and no exact operator does better—we built Provenance-Guided Selective Replay, a cross-layer contract matching a full reset under every audit at a fraction of its cost.

Limitations. The guarantee covers reconstructible runtime state, not model parameters (Liu et al. 2025), committed side effects (A3), or correlates of z ; recomputation cannot drop below $T - \tau + 1$. Assumption (A2) fixes post- τ observations, so replay covers snapshot-replayable tool returns and memory injections but not human turns that would have differed. Audits are string-based apart from the behavioral suite, and the distributional result is non-significance, not equivalence.

References

- Beurer-Kellner, L.; Buesser, B.; Crețu, A.-M.; Debenedetti, E.; Dobos, D.; Fabian, D.; Fischer, M.; Froelicher, D.; Grosse, K.; Naeff, D.; Ozoani, E.; Paverd, A.; Tramèr, F.; and Volhejn, V. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. *arXiv preprint arXiv:2506.08837*.
- Bourtole, L.; Chandrasekaran, V.; Choquette-Choo, C. A.; Jia, H.; Travers, A.; Zhang, B.; Lie, D.; and Papernot, N. 2021. Machine Unlearning. In *IEEE Symposium on Security and Privacy (S&P)*.
- Cao, Y.; and Yang, J. 2015. Towards Making Systems Forget with Machine Unlearning. In *IEEE Symposium on Security and Privacy (S&P)*.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In *arXiv preprint arXiv:2504.19413*.
- Costa, M.; Köpf, B.; Kolluri, A.; Paverd, A.; Russinovich, M.; Salem, A.; Tople, S.; Wutschitz, L.; and Zanella-Béguelin, S. 2025. Securing AI Agents with Information-Flow Control. *arXiv preprint arXiv:2505.23643*.
- Debenedetti, E.; Shumailov, I.; Fan, T.; Hayes, J.; Carlini, N.; Fabian, D.; Kern, C.; Shi, C.; Terzis, A.; and Tramèr, F. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813*.
- Debenedetti, E.; Zhang, J.; Balunović, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS), Datasets and Benchmarks Track*.
- Eldan, R.; and Russinovich, M. 2023. Who’s Harry Potter? Approximate Unlearning in LLMs. *arXiv preprint arXiv:2310.02238*.
- European Parliament and Council of the European Union. 2016. Regulation (EU) 2016/679: General Data Protection Regulation, Article 17 (Right to Erasure). Official Journal of the European Union.
- Gim, I.; Chen, G.; Lee, S.-s.; Sarda, N.; Khandelwal, A.; and Zhong, L. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems (MLSys)*.
- Grattafiori, A.; et al. 2024. The Llama 3 Herd of Models. *arXiv preprint arXiv:2407.21783*.
- Greshake, K.; Abdelnabi, S.; Mishra, S.; Endres, C.; Holz, T.; and Fritz, M. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*.
- Jia, Y.; Liu, Y.; Shao, Z.; Jia, J.; and Gong, N. Z. 2026. PromptLocate: Localizing Prompt Injection Attacks. In *IEEE Symposium on Security and Privacy (S&P)*.
- Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D. S.; de las Casas, D.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825*.
- Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C. H.; Gonzalez, J. E.; Zhang, H.; and Stoica, I. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.
- Liu, S.; Yao, Y.; Jia, J.; Casper, S.; Baracaldo, N.; Hase, P.; Yao, Y.; Liu, C. Y.; Xu, X.; Li, H.; Varshney, K. R.; Bansal, M.; Koyejo, S.; and Liu, Y. 2025. Rethinking Machine Unlearning for Large Language Models. *Nature Machine Intelligence*.
- Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, F.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2024. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. *arXiv preprint arXiv:2408.04682*.
- Maini, P.; Feng, Z.; Schwarzschild, A.; Lipton, Z. C.; and Kolter, J. Z. 2024. TOFU: A Task of Fictitious Unlearning for LLMs. In *Conference on Language Modeling (COLM)*.
- Nous Research. 2026. Hermes Agent: A Self-Hosted, Long-Running Autonomous Agent. <https://github.com/NousResearch/hermes-agent>. Accessed July 2026.
- Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. In *arXiv preprint arXiv:2310.08560*.
- Steinberger, P. 2026. OpenClaw: An Open-Source Personal AI Agent. <https://github.com/openclaw/openclaw>. Accessed July 2026.
- Wallace, E.; Xiao, K.; Leike, R.; Weng, L.; Heidecke, J.; and Beutel, A. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv preprint arXiv:2404.13208*.
- Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *International Conference on Learning Representations (ICLR)*.
- Yang, A.; et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.
- Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; Yu, C. H.; Cao, S.; Kozyrakis, C.; Stoica, I.; Gonzalez, J. E.; Barrett, C.; and Sheng, Y. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.